# SIMATS ENGINEERING

# CSA6303- Threat Intelligence and Network Security

# A1-Malware Analysis Assignment

Assessment Questions

| | | | |
|---|---|---|---|
| **Subject** | Threat Intelligence | **Total Marks** | 50 |
| **Questions** | 10 (all compulsory) | **Marks per Question** | 5 |
| **REg.No** | 192312353 | **Name** | VELUGU KIRAN BOJA RAJU |
| **Faculty** | Dr.V.Parthipan | **Slot** | A |

*Instructions: Answer all questions. Each question carries 5 marks. Support answers with examples and diagrams where appropriate.*

***CO3:*** Perform threat analysis using intelligence models and utilize industry tools for monitoring and detection.

***Code of Conduct:***

*I VELUGU KIRAN BOJA RAJU/192312353 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

## Signature of Student:V.Kiran

## Questions :

**1. Malware Analysis Using Static Techniques:** A security analyst receives a suspicious executable file attached to a phishing email. Before running the file, the analyst wants to examine its properties, file hash, strings, and imported functions. Explain how static malware analysis can help identify suspicious characteristics and assess the potential risk without executing the file.

**2. Malware Analysis Using Dynamic Techniques:** An organization suspects that an employee has downloaded a malicious program that may modify system files and communicate with an external server. Explain how dynamic malware analysis can be performed in an isolated virtual environment to observe the program's behavior and identify potential Indicators of Compromise (IoCs).

**3. Malware Investigation Using a Threat Intelligence Platform:** A company's security team discovers a suspicious file on an employee's computer. The file hash appears in a threat intelligence platform associated with a known malware family. Explain how the analyst can use threat intelligence sources to investigate the file, assess the reliability of the information, and recommend suitable containment measures.

**4. Malware Detection Through Data Collection and Correlation:** A university detects unusual processes on several computers, suspicious DNS requests, and unexpected outbound network connections. Explain how an analyst can collect and correlate endpoint logs, DNS records, firewall logs, and file hashes to determine whether the computers are infected with malware.

**5. Malware Investigation Using SIEM Tools:** A financial institution's Security Information and Event Management (SIEM) system generates alerts about suspicious process execution, repeated failed logins, and unusual network connections from the same workstation. Describe how the security analyst can use SIEM capabilities to correlate these events, investigate the possible malware infection, and prioritize the incident.

**6. Malware Detection and Log Analysis Using Splunk:** A company uses Splunk to monitor its network and endpoint logs. The security team suspects that malware is attempting to contact an external command-and-control server. Explain how Splunk can be used to search relevant logs, identify suspicious communication patterns, correlate events, and document evidence of possible malware activity.

**7. Malware Investigation Using the ELK Stack:** An e-commerce company uses the ELK Stack (Elasticsearch, Logstash, and Kibana) to monitor web server logs, firewall records, and endpoint events. The security team notices repeated connections to suspicious domains and unusual file activity. Explain how the ELK Stack can support malware investigation through log collection, processing, searching, visualization, and correlation.

**8. Threat Hunting for Malware Activities:** An organization's security team suspects that malware has infected an employee workstation but has not received any antivirus alerts. The workstation shows unusual CPU usage, unfamiliar processes, and connections to unknown external IP addresses. Describe a threat-hunting approach to investigate the workstation, identify suspicious behaviors, collect relevant IoCs, and recommend remediation measures.

**9. Comparing Malware Analysis Tools Used in the IT Industry:** A cybersecurity team needs to investigate a suspicious file and determine whether it contains malicious code. Compare static analysis tools, dynamic analysis tools, network traffic analyzers, and threat intelligence platforms in terms of their purpose and usefulness. Recommend an appropriate combination of tools for conducting a malware investigation.

**10. Malware Analysis Report and Incident Response:** A company discovers a suspicious executable on an employee's computer. Initial analysis identifies an unfamiliar process, a modified system configuration, and communication with a suspicious external domain. Prepare an outline for a malware analysis report that includes file information, analysis methodology, observed behavior, IoCs, threat intelligence findings, risk assessment, and recommended containment and recovery actions.

# ANSWERS

## 1. Malware Analysis Using Static Techniques

Static malware analysis is the examination of a suspicious file without executing it. It helps an analyst understand the file's structure and identify signs of malicious intent while reducing the risk of infecting a computer. In this case, the executable attached to a phishing email should be handled in a controlled analysis environment and must not be opened on a normal workstation.

First, record basic file properties such as filename, file size, file type, creation or modification timestamps when available, and digital signature information. Check whether the file extension matches its actual format. A file named as a PDF but identified as a Windows executable is suspicious.

Second, calculate cryptographic hashes such as SHA-256. The hash acts as a fingerprint that can be compared with known malware samples or threat-intelligence databases. A hash match can provide useful evidence, although a hash that is not known does not prove that the file is safe.

Third, extract readable strings from the file. Strings may reveal URLs, IP addresses, domain names, file paths, registry keys, error messages, or commands. Obfuscated or packed malware may hide these strings, so the absence of suspicious strings is not proof of benign behaviour.

Fourth, inspect imported functions and libraries. Functions related to process creation, file modification, registry changes, network communication, memory manipulation, or privilege changes may indicate capabilities that require closer review. These imports are clues rather than conclusive proof because legitimate software may use the same functions.

Fifth, inspect the file's headers, sections, metadata, and digital signature using tools such as PEStudio, Detect It Easy (DIE), or other approved PE-analysis utilities. A missing or invalid signature, unusual section names, suspicious entropy, packing indicators, or inconsistent metadata can increase suspicion.

The analyst should combine these findings with the email sender, message context, and reputation information from trusted threat-intelligence sources. Static analysis is quick and repeatable, but it may not reveal behaviour hidden by encryption, packing, or runtime checks. The result should include the file hash, observations, confidence level, and recommendation to quarantine or escalate the file if risk is high.

Example: An email attachment has an executable file type despite a document-like name, contains a URL to an unfamiliar domain, and imports functions associated with network communication and process creation. These findings justify treating it as suspicious and proceeding with controlled dynamic analysis.

## 2. Malware Analysis Using Dynamic Techniques

Dynamic malware analysis observes a program while it runs in a controlled and isolated environment. It is useful for identifying actions that may not be visible through static inspection, such as file changes, registry modifications, process creation, DNS requests, and communication with external servers.

Begin by preparing a dedicated virtual machine or malware-analysis sandbox that contains no sensitive information or production credentials. Take a clean snapshot, isolate or tightly control network access, and ensure the environment cannot connect freely to the organization's internal network. Use simulated services or a controlled network when possible.

Before execution, record the sample's filename, size, SHA-256 hash, and initial system state. Start monitoring tools before running the sample. Process Monitor can record file and registry activity; Process Explorer can show process relationships; Windows Event Logs or Sysmon can record process creation and network-related events; and Wireshark can capture network traffic.

Run the sample only inside the isolated environment and observe its behaviour over an appropriate period. Record child processes, files created or deleted, registry keys changed, services or scheduled tasks created, security settings modified, and attempts to gain privileges. Look for persistence mechanisms and suspicious use of scripting tools.

Monitor network activity for DNS queries, repeated connections, unusual ports, HTTP requests, TLS connections, and communication with unfamiliar IP addresses or domains. If the sample appears to contact a command-and-control server, record the destination, timestamps, protocol, and connection pattern. Do not allow uncontrolled access to live malicious infrastructure.

After execution, compare the system state with the baseline and save relevant logs, screenshots, packet captures, and process details. Extract Indicators of Compromise (IoCs), such as file

hashes, domains, IP addresses, URLs, filenames, registry paths, mutexes, and process names. Restore the virtual machine to its clean snapshot after the investigation.

Dynamic analysis can reveal actual behaviour, but malware may detect virtual machines, delay execution, or behave differently depending on its environment. Therefore, combine dynamic findings with static analysis and threat intelligence. Example: A sample creates a scheduled task, drops a file in a temporary directory, and repeatedly resolves an unfamiliar domain. These observations provide behavioural evidence and IoCs for detection.

## 3. Malware Investigation Using a Threat Intelligence Platform

A threat intelligence platform (TIP) collects, organizes, and enriches information about threats, including file hashes, malicious domains, IP addresses, URLs, malware families, attack techniques, and related campaigns. If a suspicious file's hash is associated with a known malware family, the analyst can use that information to guide investigation and containment.

First, calculate the file's SHA-256 hash and search for it in the organization's TIP and trusted external sources. Review the malware-family name, first-seen and last-seen dates, associated indicators, observed behaviours, targeted systems, and any known attack techniques. Search for related domains, IP addresses, file names, and hashes in endpoint, DNS, proxy, and firewall logs.

Second, assess reliability. Consider the source's reputation, how the indicator was collected, how recently it was updated, the confidence score, and whether multiple independent sources agree. Check whether the hash is an exact match and whether the intelligence is relevant to the organization's environment. Shared hosting, reused infrastructure, stale data, or incorrect submissions can create false positives.

Third, correlate the intelligence with local evidence. Determine whether the file was executed, which user launched it, whether it created other files or processes, and whether the device contacted associated domains or IP addresses. Search other endpoints for the same hash and related indicators to identify the scope of the incident.

If the file is confirmed or strongly suspected to be malicious, quarantine it using approved endpoint tools, isolate affected devices when necessary, block verified malicious indicators at network controls, and preserve evidence. Reset exposed credentials if credential theft is suspected, remove persistence, patch exploited weaknesses, and monitor for related activity.

Avoid blocking broad shared infrastructure without validation because this may disrupt legitimate services.

The analyst should document the indicator, source, confidence, evidence, affected assets, actions taken, and remaining uncertainty. Threat intelligence supports decisions but should not replace local investigation. Example: A hash is linked by several reliable sources to a credential-stealing malware family, and EDR logs show that the file ran before an unusual outbound connection. The combined evidence supports urgent containment and credential-risk assessment.

## 4. Malware Detection Through Data Collection and Correlation

Data collection and correlation combine information from endpoints, DNS services, firewalls, and file analysis to determine whether suspicious events are related. In the university scenario, unusual processes, suspicious DNS requests, and unexpected outbound connections across several computers may indicate a shared malware infection or coordinated activity.

Collect endpoint telemetry, including process creation, parent-child process relationships, command-line arguments, file modifications, persistence changes, antivirus/EDR alerts, and user logins. Windows Sysmon, endpoint security agents, and operating-system event logs can provide useful details. Record hostnames, usernames, timestamps, process IDs, and file hashes.

Collect DNS records showing queried domains, query times, response addresses, and requesting hosts. Look for newly observed domains, unusually long or random-looking domain names, frequent failed lookups, and domains associated with known malicious activity. DNS anomalies alone are not proof of malware and must be checked against normal application behaviour.

Collect firewall and network records, including source and destination IP addresses, ports, protocols, connection times, bytes sent and received, and allowed or blocked actions. Determine whether several computers contacted the same unfamiliar external host, whether connections repeated at regular intervals, and whether large or unusual transfers occurred.

Calculate SHA-256 hashes for suspicious files and compare them with internal records and trusted threat-intelligence sources. Correlate events using time, hostname, user, process ID, IP address, domain, and hash. For example, a suspicious process starts on three computers, each

device queries the same domain, and all three connect to the same external IP address shortly afterwards. This sequence strengthens the case for a common threat.

Use a SIEM such as Splunk or Elastic Security to search and correlate the collected data. Build an event timeline, identify the earliest affected host, and search for the same indicators across the network. Validate the findings to avoid false positives caused by legitimate software updates or approved services.

If infection is likely, isolate affected devices as appropriate, quarantine malicious files, block verified indicators, preserve logs, and reset credentials if needed. Patch the entry point and monitor for recurrence. Document data sources, the correlation method, findings, confidence, and remediation. Correlation is powerful because it links individual clues into a more complete incident picture.

## 5. Malware Investigation Using SIEM Tools

A SIEM system centralizes logs from endpoints, servers, identity services, firewalls, and other security controls. When it detects suspicious process execution, repeated failed logins, and unusual network connections from one workstation, analysts can correlate these events to assess whether malware or account compromise is occurring.

First, review the alert details, severity, timestamp, affected host, username, source and destination addresses, and detection rule. Confirm that the relevant logs are available and that system clocks are synchronized. Expand the search window to examine activity before and after the alert.

Second, correlate events. Repeated failed logins may indicate password guessing or a user entering incorrect credentials. A subsequent successful login, followed by an unfamiliar process and an external connection, may indicate that an account was compromised and used to run malware. Correlate process creation with DNS, firewall, proxy, EDR, and authentication records.

Third, enrich indicators with threat intelligence. Check suspicious domains, IP addresses, URLs, and file hashes against reliable sources. Examine the process path, parent process, command line, signature, and file hash using endpoint tools. Search the SIEM for the same indicators on other systems to determine whether the incident has spread.

Prioritize the incident according to confidence, asset criticality, account privileges, evidence of active communication, possible data exposure, and number of affected systems. A workstation contacting a confirmed malicious domain while running an unknown process should generally receive greater priority than an isolated low-confidence failed-login alert.

Follow the incident-response playbook. Preserve evidence, isolate the workstation when appropriate, disable or reset compromised accounts, block verified malicious destinations, and escalate to the incident-response team. Record the timeline, related alerts, investigation queries, evidence, risk rating, and actions taken. SIEM alerts should be validated because legitimate administrative tasks can also trigger detections.

## 6. Malware Detection and Log Analysis Using Splunk

Splunk can ingest, index, search, and visualize security events from endpoint agents, DNS servers, firewalls, proxies, and other systems. It can help identify whether a workstation is repeatedly contacting a possible command-and-control (C2) server and correlate those connections with process execution or other suspicious activity.

Begin by identifying relevant data sources and indexes. Search endpoint logs for process creation and suspicious file activity; DNS logs for unusual domain lookups; and firewall or proxy logs for outbound connections. Useful fields include timestamp, host, user, process name, file hash, query name, destination IP, destination port, bytes transferred, and action.

Search for patterns such as repeated connections to the same external address, regular connection intervals, connections to newly observed domains, unusual ports, and outbound traffic shortly after an unfamiliar process starts. Use time charts and tables to compare connection frequency, destination, and data volume with normal behaviour. These patterns are indicators for investigation, not proof by themselves.

Correlate events by hostname, timestamp, user, domain, IP address, and file hash. For example, endpoint logs may show an unknown executable starting at 10:03, DNS logs may show a lookup for a suspicious domain at 10:04, and firewall logs may show a connection to the resolved IP at 10:04. This sequence can support a hypothesis of C2 activity.

Enrich the domain, IP address, and file hash with threat intelligence, then validate the result using EDR or packet analysis. Save relevant search results, timestamps, event identifiers,

screenshots or exports where permitted, and the queries used. Ensure the evidence is protected and the report distinguishes observed facts from assumptions.

If activity is confirmed or strongly suspected, follow the incident-response process to isolate the host, block verified indicators, investigate persistence, and check for other affected devices. Splunk can also support detection-rule improvements and dashboards for ongoing monitoring. The reliability of the result depends on log coverage, correct field extraction, synchronized time, and well-tuned searches.

## 7. Malware Investigation Using the ELK Stack

The ELK Stack consists of Elasticsearch, Logstash, and Kibana. Elasticsearch stores and searches indexed events, Logstash collects and processes data, and Kibana provides dashboards, visualizations, and investigation interfaces. Together, they can help analysts investigate suspicious domains and unusual file activity across web servers, firewalls, and endpoints.

First, collect logs from web servers, endpoint agents, DNS services, firewalls, and other relevant systems. Data can be forwarded through agents or ingestion pipelines. Logstash parses, filters, enriches, and normalizes fields such as timestamps, hostnames, usernames, source and destination IP addresses, domain names, process names, and file hashes.

Second, index the processed events in Elasticsearch. Analysts can search events over a selected time range, filter by host or indicator, and compare related records. Kibana dashboards can display DNS requests, outbound connection counts, suspicious file events, and event timelines. Visualizations can reveal spikes, repeated connections, or multiple devices contacting the same destination.

Third, correlate logs. For example, endpoint logs show an unfamiliar executable being created, DNS records show a query to a suspicious domain, and firewall logs show an outbound connection to the resolved IP. Correlation by timestamp, host, domain, and hash can help determine whether the events form a single sequence. Search for the same indicators across all available data sources to assess scope.

Fourth, enrich indicators with threat intelligence and validate suspicious activity with endpoint investigation or packet analysis. Review data quality, missing fields, duplicate events, and time

synchronization. A dashboard is useful for visibility, but analysts must inspect underlying events before reaching a conclusion.

Finally, document the timeline, queries, visualizations, indicators, affected assets, and confidence level. If malware is likely, isolate affected endpoints, block confirmed malicious destinations, preserve evidence, and remediate the cause. ELK's usefulness depends on correct ingestion pipelines, adequate retention, access controls, and well-designed dashboards and detection rules.

## 8. Threat Hunting for Malware Activities

Threat hunting searches proactively for malicious activity even when antivirus software has not generated an alert. Unusual CPU usage, unfamiliar processes, and connections to unknown external IP addresses can indicate malware, cryptomining, unwanted software, or legitimate but unexpected workloads. The analyst should test a hypothesis and validate findings using multiple sources.

Begin by recording the workstation's normal CPU and memory usage, active processes, network connections, and usual destinations. Review process names, paths, digital signatures, parent-child relationships, command lines, and file creation times. Investigate processes running from temporary folders, user profile directories, or unusual locations, especially when they are unsigned or recently created.

Inspect operating-system and EDR logs for persistence, scheduled tasks, new services, startup entries, suspicious scripts, privilege changes, and security-tool tampering. Review authentication logs for unfamiliar logins or privilege use. Check whether the process is associated with approved software or a recent update before classifying it as malicious.

Examine network connections using endpoint telemetry, firewall records, DNS logs, and tools such as Wireshark or Zeek. Record destination IPs, domains, ports, protocols, connection times, and traffic volume. Repeated periodic connections, suspicious DNS requests, or communication with a known malicious address may indicate C2 activity.

Collect IoCs such as SHA-256 hashes, filenames, file paths, domains, IP addresses, URLs, registry paths, and process names. Compare them with trusted threat-intelligence sources and search other devices for the same indicators. Correlate process execution with network activity and user logins to build a timeline.

If compromise is likely, preserve evidence and isolate the workstation where appropriate. Quarantine malicious files, block confirmed malicious indicators, reset credentials if they may have been exposed, remove persistence, and patch the exploited weakness. Restore from a trusted backup if required. Continue monitoring for recurrence and document the investigation, confidence, impact, and lessons learned.

## 9. Comparing Malware Analysis Tools Used in the IT Industry

Malware investigations are more reliable when several tool categories are used together. Static tools inspect files without running them; dynamic tools observe behaviour during execution; network analyzers examine communication; and threat-intelligence platforms provide context about indicators and known threats.

Static analysis tools: Utilities such as PEStudio, Detect It Easy (DIE), strings tools, and hash utilities examine file metadata, hashes, headers, imports, sections, signatures, and readable strings. They are fast and useful for initial triage without executing the file. However, packing, encryption, and obfuscation can hide important details, and imported functions do not prove malicious intent.

Dynamic analysis tools: Sandboxes and isolated virtual machines allow analysts to observe process creation, file and registry changes, persistence, and network activity. Tools such as Process Monitor, Process Explorer, Sysmon, and approved malware sandboxes provide behavioural evidence. Dynamic analysis can reveal actions missed by static analysis, but malware may detect the environment or delay execution.

Network traffic analyzers: Wireshark supports detailed packet inspection, while Zeek produces structured network metadata and protocol logs. These tools help identify DNS queries, suspicious destinations, unusual protocols, repeated connections, and possible data exfiltration. Encryption can limit payload visibility, and a packet capture may not cover every relevant connection.

Threat intelligence platforms: MISP, OpenCTI, and other trusted intelligence services organize indicators, connect them to malware families or campaigns, and provide context about known infrastructure. They help prioritize investigation and create detection rules. Their results must be evaluated for source reliability, confidence, freshness, and false positives.

Recommended combination: Begin with static analysis to calculate hashes and inspect file properties. Use threat intelligence to enrich the indicators. If the file remains suspicious, run it in an isolated sandbox while recording endpoint changes and network activity. Use Wireshark or Zeek to examine communications, then search SIEM and endpoint logs for the extracted IoCs across the organization. This layered approach combines file characteristics, observed behaviour, network evidence, and external context.

Example: Static analysis identifies a suspicious URL in an executable; threat intelligence associates the domain with a malware family; sandbox execution shows a scheduled task and repeated DNS queries; and firewall logs confirm that an employee workstation contacted the same domain. The combined evidence supports a more confident assessment than any single tool alone.

## 10. Malware Analysis Report and Incident Response

A malware analysis report provides a structured record of what was examined, how it was analyzed, what behaviour was observed, which systems may be affected, and what actions are recommended. It supports incident response, management decisions, evidence preservation, and future detection improvements.

1. Incident summary and scope: Record the incident identifier, date and time, analyst, discovery method, affected user and device, business impact, and purpose of the investigation. State whether the report is preliminary or final.

2. File information: Include filename, file path, file size, file type, SHA-256 hash, digital signature status, timestamps when available, and source of the file. Preserve the original sample according to organizational evidence-handling procedures.

3. Analysis methodology and tools: Describe the static analysis, dynamic analysis, threat-intelligence checks, endpoint investigation, and network analysis performed. Name the tools and relevant versions when known, identify the isolated environment used, and note any limitations.

4. Observed behaviour: Document the unfamiliar process, parent process, command-line arguments, files created or modified, registry or system configuration changes, persistence mechanisms, privilege changes, and attempted security-control modifications. Include the

observed connection to the suspicious external domain, relevant IP address, port, protocol, and timestamps.

5. Indicators of Compromise: List confirmed and suspected hashes, filenames, file paths, domains, URLs, IP addresses, registry paths, process names, and other useful indicators. Distinguish indicators directly observed in the environment from those obtained from threat intelligence.

6. Threat intelligence findings: Record the sources consulted, associated malware family if supported, confidence and freshness of the intelligence, related indicators, and any conflicting information. Do not treat a single unverified reputation result as conclusive proof.

7. Risk assessment and scope: Explain the likelihood of compromise, possible data or credential exposure, privileges involved, criticality of the affected device, evidence of lateral movement, and whether other devices show the same indicators. Clearly identify confirmed facts, probable findings, and unresolved questions.

8. Containment, eradication, and recovery: Recommend isolating the affected workstation where appropriate, quarantining the executable, blocking verified malicious destinations, disabling or resetting potentially compromised accounts, removing persistence, patching the entry point, and restoring systems from trusted sources if required. Preserve evidence before destructive remediation when possible.

9. Monitoring and prevention: Recommend searching the environment for all identified IoCs, tuning SIEM and EDR detections, strengthening email filtering, enforcing multi-factor authentication, applying least privilege, patching vulnerable software, and improving outbound network controls. Assign owners and deadlines for follow-up actions.

10. Evidence and conclusion: Attach or reference relevant log excerpts, event IDs, timelines, screenshots, packet captures, and investigation queries where permitted. Explain the overall conclusion, confidence level, remaining uncertainty, and criteria for closing the incident.

Example conclusion: The workstation executed an unfamiliar file, modified system configuration, and contacted a suspicious domain. If these observations are supported by endpoint and network evidence, the incident should be treated as a likely compromise pending final validation. The workstation should be contained, credentials assessed, related systems searched for the same IoCs, and recovery verified before the incident is closed.